K.H.U. Faculty of Engineering and Natural Sciences

FENS 401-402 BACHELORS PROJECT PROPOSAL

ADAPTIVE AGENTIC RAG FRAMEWORK FOR REASONING OVER HETEROGENEOUS DATA SOURCES

Project Advisor(s): Assoc. Prof. Dr. Taner Arsan

Project Team:

Erk Demirel[1]

Emir Özen[1]

Mehmet Ekin Ay[1]

Volkan Erdoğan[1]

Submission Date: 30.12.2025

[1] Computer Engineering, Engineering and Natural Sciences Faculty, Kadir Has University

ABSTRACT:

Effective decision taking processes usually require the usage of both structured data such as SQL tables and unstructured data such as PDF reports. But the current Retrieval-Augmented Generation (RAG) approaches struggle with efficiency while trying to integrate these two data types. This project proposes an adaptive agentic RAG framework. The proposed system uses a decomposition agent that breaks down the question into meaningful sub-tasks instead of answering it directly. It also uses a RAG agent and directs each sub-task to suitable data processing component (structured, unstructured). For structured data, an adaptive dual-path pipeline is designed. First, the system checks a vector cache that stores previous queries and in the case of a cache hit a fast and low-cost answer is generated. If there is not a cache hit, the system uses a TableRAG and LangChain based agent to explore the database schemas dynamically. Unstructured data are processed by a FAISS based vector search. Finally, a synthesis agent, combines two answers generated by both sources to generate a consistent answer. This study demonstrates that adaptive switching between cache-based rapid responses and agent-based deep reasoning effectively reduces response time. It also improves the reliability of results across heterogeneous data sources.

AIM:

Retrieval-Augmented Generation (RAG), became a widely used approach to make the Large Language Models (LLMs) more reliable and also support them with verifiable external data [1]. But the current RAG systems are usually limited with a single data type: they either pull data from unstructured data such as text or only generate SQL queries for the structured data such as databases. But real user prompts usually require the handling of both structured and unstructured data at the same time [2]. This situation shows the mismatch issue between user intention and system capabilities and thus increases the need for more adaptive RAG structures that can handle both data types [3].

This project's aim is to develop an adaptive Agent-based RAG system that can work with both structured and unstructured data. The RAG system uses a mechanism that separates the user prompt into parts and then decides where to get the necessary information based on each separate user prompt. The separated user prompt will be treated as a heterogeneous data; prompt part that requires structured data retrieval will be forwarded to the structured data retrieval pipeline whereas the prompt part that requires unstructured data will be forwarded to the unstructured data retrieval section. In the end a synthesis agent will combine the both answers from both pipelines for the best answer. Furthermore, the fast retrieval of the text will be provided with FAISS based vector search [4] by cached SQL query patterns and schema focused TableRAG approach [5].

Instead of trying to achieve state-of-the-art performance, this project focuses on the feasibility of the architecture. Especially the adaptive routing, multi-stepped reasoning and output combining will be analyzed to observe how these processes affect system behavior, latency and consistency.

1 Research Questions & Objectives

1.1 Specific Objectives to Achieve This Aim

The project focuses on the following objectives:

1.1.1 Primary Objectives

1. **Implement Semantic Intent Classification:** The first primary objective of the project is to develop a Router Agent using a Zero-Shot LLM classifier that analyzes user prompts and routes them to the appropriate subsystem (SQL-only, Text-only, or Hybrid) based on semantic intent.
2. **Integrate a "Dual-Path" SQL Workflow:** The second primary objective of the project is to successfully orchestrate a hybrid pipeline that switches between a "Fast Track" and a "Reasoning Track".

1.1.2 Secondary Objectives

1. **Integrate TableRAG for Schema Pruning:** The first secondary objective of the project is to adapt and integrate the open-source TableRAG framework to dynamically select relevant table schemas from the database. Through this, reducing the context window usage for the LangChain Agent [6].
2. **Develop an Interactive Verification Interface:** The second and the final objective of this project is to design and implement a user interface that allows users to input natural language queries and visualize the system's process on retrieving.

1.2 Research Questions

The main purpose of this study is to determine if it’s feasible to create an Adaptive Agentic RAG system that can handle both structured and unstructured data by forwarding it to the corresponding pipeline. For this aim we have three goals: calculating the effectivess of the caching approach on the latency, testing the accuracy of the semantic intent classification on the system and measure the effect of schema pruning on SQL

generation reliability. The research questions (RQs) and their research strategies (RSs) are shown on the list below:

RQ1: Does the caching approach effectively reduce system latency whenever there is a recurring query compared to the reasoning path?
RS1: We will use a vector-based "Fast Track" by using FAISS to store and retrieve successful SQL patterns. Following that, we will use a dataset consisting of both repetitive and novel queries, and then measure the latency difference between these two within the RAG system, which will use the cache and the reasoning pipelines respectively.

RQ2: How accurately can the Router Agent separate the prompt and then classify them to sub-tasks that require structured data queries versus sub-tasks that require semantic text search?
RS2: The performance of the Router Agent will be evaluated with a controlled test set that consists of 30 mixed queries (structured, unstructured and hybrid) to determine how accurate the classification is and if it can reliably organize the workflow without human intervention.

RQ3: Does integrating TableRAG's schema pruning significantly reduce the rate of syntax errors in generated SQL compared to feeding the full database schema?
RS3: We will use the TableRAG framework for a preliminary semantic table search that filters table schemas before generation. After that we will compare the generated SQL queries on how accurate they are versus to those generated with full schema knowledge.

1.3 Parameters for Checking the Aim

The system will be evaluated using the following feasibility and behavior metrics:

1. Latency Differential (System Behavior): We will measure the time difference between the "Fast Track" (Few-Shot) and "Reasoning Track" (Agentic) paths for our first aim.

2. Router Classification Accuracy: The second parameter for checking the aim is the ability of the Router Agent to correctly classify query intent (SQL/Text/Hybrid) on a controlled test set of 30 queries.

3. Functional Integration: The third parameter for checking the aim is the successful demonstration of at least 5 End-to-End flows where a single prompt triggers both PDF retrieval and SQL execution, producing a unified natural language response.

4. Execution Success Rate: The fourth and the final parameter for checking the aim is the percentage of generated SQL queries that execute without syntax errors on a valid test set.

METHODOLOGY:

1. Information Retrieval

In today's data environment data types split into two main categories: unstructured and structured data. Unstructured data (such as reports, documents, emails, etc.) is not arranged by a schema or model. Structured data is data that is stored in relational databases. Structured data is typically tabular and organized with clear attributes. Traditionally, these two main data categories have been managed by different concepts. Information Retrieval (IR) methods used for analysis of textual data and searching text. Moreover, Database Management Systems (DBMS) for querying structured data. However, the decision-making processes of organizations do not quite fit neatly into one category. An example query such as "Compare our European subsidiaries' revenue growth rate with market sentiment as described in their regional performance report" means that you are mixing numeric data extracted from SQL tables with information entailed in text documents. This represents a problem in designing a system that would offer comprehensive and intelligent answers.

This project's main aim is to present a detailed architectural plan for a Hybrid Agentic Retrieval-Augmented Generation (RAG) system that can reason over different types of data simultaneously. Instead of following the "traditional" model of statis "retrieve then generate" algorithm used in early Large Language Model (LLM) systems, this project focuses on the newer concept of agentic RAG.

To understand the architecture proposed in this study, we primarily focused on two technological concepts: Large Language Models (LLM) and Retrieval-Augmented Generation (RAG).

Large Language Models are large scale neural networks based on transformers architecture. LLMs are able to carry out tasks related to natural language processing (NLP), including text completion, translation, reasoning, and summarization [1]. The ability of LLMs to store reliable data stems from their self-attention processes, which are enhanced by parameter amount and unsupervised pretraining [2]. In addition, LLMs can catch long-range similarities and relationships in text. Despite their capabilities, LLMs are limited by their training data. Furthermore, LLMs cannot update their knowledge dynamically without using any external services [2].

Retrieval-Augmented Generation (RAG) is a framework which allows LLMs to integrate with external document retrieval with a generation pipeline. This allows the model to include domain-specific or current material from a database before producing a response [3]. By using dense or sparse similarity search methods to retrieve similar documents, these documents can be provided as context to the model during generation. These methods improve the accuracy of responses and decrease hallucinations in knowledge-intensive tasks [3, 7].

2.1 Unstructured Retrieval: From Lexical to Neural Models

Finding meaningful text segments from large collections of non-relational data, such PDFs and reports. In recent years, this field has gone through a fundamental change, moving from heuristic methods to complicated deep learning formulations. The trajectory is examined in the following subsections, which highlight the particular advantages and disadvantages of each method.

2.1.1 Lexical Retrieval: BM25 as the Classical Baseline

Okapi BM25 algorithm is one of the industry standards for first-stage retrieval in information systems. BM25 described in the probabilistic relevance framework, as the algorithm assigns term frequency (tf), inverse document frequency (idf), and a length-normalization part to each document to correct for document verbosity [8]. Later analysis on the algorithm mentions BM25's stability, parameter robustness, and high performance tested on a variety of retrieval benchmarks [9].

Despite these strengths, BM25 is still a lexical retrieval method. The algorithm misses deep semantic relationships because of requirement of precise match of queries and keywords. In hybrid reasoning scenarios, especially in contemporary ones, this problem acts as a major drawback. For instance, a search query consisting of "fiscal tightening measures," which can match documents with "budget cuts" or "interest rate hikes," but a lexical approach. BM25 lacks the semantic capabilities to fill this gap and negatively affects recall performance for semantically similar documents [8, 10].

On the other hand, as seen in several studies assessing classical probabilistic approaches, a lack of vocabulary matches has long been a major reason for failures in downstream reasoning tasks relying on LLMs [9, 10].

2.1.2 Neural Dense Retrieval: DPR and Semantic Representation Learning

Dense Passage Retrieval (DPR) marked a significant change by introducing a dual-encoder architecture that encoded texts and queries into a common embedding space [11]. The approach can capture semantic similarity beyond lexical overlap since relevance is calculated using dot-product similarity. Dense semantic retrieval became a new benchmark as the result of open-domain QA tests, DPR outperformed BM25 by 9%–19% in top 20 retrieval accuracy [11].

A recent test has also demonstrated that DPR still has a competitive performance in conversation-oriented and multi-turn-based scenarios, in which queries are often articulated in different ways, possibly calling for a form of semantic generalization. This test has further validated that DPR keeps a high-level performance in query-based retrieval even with limited fine-tuning for conversational reformulation, solidifying DPR's role as a reliable all-purpose semantic retriever for RAG models [12]. While this reliance of DPR on a supervised dataset like MSMARCO presents a problem where domain-based questions and passages are potentially sparse and expensive.

2.1.3 Unsupervised Dense Retrieval: Contriever, GTR, HyDE

To overcome supervision bottlenecks, unsupervised methods such as Contriever rely on contrastive learning over raw text without annotations [13]. Contriever trains directly on raw text instead of using manually classified question-passage pairings. It does this by creating positive pairs using minimal augmentations, such clipping or masking various parts of the same document and considering all other documents in the batch as negatives. In comparison to supervised dense retrievers, this enables the model to learn multipurpose semantic representations without any supervision and achieve competitive retrieval efficiency with no annotation effort. Because of this feature, Contriever is especially well suited for business settings where producing domain-specific labeled data is difficult or costly [14].

Furthermore, HyDe (Hypothetical Document Embeddings) provides a generative model for zero-shot retrieval by asking a language model to imagine a hypothetical document as a response in place of embedding a query [14]. This document can then be used for dense retrieval as it enhances the query's semantic embedding. As outlined in the HyDE implementation by DeepSet, it works as a form of query expansion to help an Agent-like model estimate what it needs before finding actual proof [15].

2.1.4 Late Interaction Models for High-Precision Retrieval: ColBERT

A late interaction architecture called ColBERT (Contextualized Late Interaction over BERT) closes the gap between effective retrieval and expressive contextual deep modeling. ColBERT uses token-level abstractions in place of single-vector document embeddings and uses MaxSim to calculate relevance by matching each query token to the most comparable document token [16]. ColBERT achieves orders of magnitude quicker inference than conventional BERT ranking algorithms while achieving retrieval accuracy comparable to cross-encoders. It regularly performs better than traditional lexical and dense baselines, especially in vast retrieval settings, according to benchmarks such MSMARCO [16]. ColBERT's function as an efficient high precision reranker in hybrid RAG systems is reinforced by further assessments that verify multi-vector post-interaction models significantly improve recall and ordering quality [17].

2.2 Structured Retrieval: The Renaissance of Text-to-SQL

A hybrid RAG system's structured component has to be able to perform accurate quantitative evaluation and integrate with relational databases. The Text-to-SQL job, which converts natural language queries into executable SQL statements, has traditionally been used to handle this capacity. Over the past five years, the field has rapidly changed. Text-to-SQL serves as the bridge for hybrid RAG, between natural language reasoning and structured enterprise data.

2.2.1 The Shift from Fine-Tuning to In-Context Learning

Because it transforms natural language searches into executable queries in SQL across relational databases, text-to-SQL is an essential part of hybrid RAG systems for structured data. In the past, early methods relied on rule-based systems that had trouble generalizing, especially across unfamiliar databases or complicated SQL structures. Spider gigantic human-labeled benchmark including more than 200 datasets in 138 fields [18]. Unlike earlier datasets, the Spider benchmark evaluates a model's ability to generalize rather than retain query patterns by differentiating across training and test databases. Because Spider contains complex SQL structures like joins, nested queries, and aggregation, it is helpful for assessing lexical parsers and modern LLM-based methods [18].

In a multi-sample critique approach single query produces multiple candidate SQL queries. A separate model then evaluates these candidates using execution results and constraint checks to select the syntactically valid and semantically correct query [19]. By allowing smaller, open-source LLMs to reach the performance of bigger corporate models, this strategy highlights the importance of iterative improvement and execution feedback. These methods have decreased the chance of generating inadequate SQL statements and proposed a feasible solution to integrate LLMs into different kinds of datasets within the enterprise systems [19].

Another study on this issue provides an overview of LLM-enhanced text-to-SQL techniques. This overview shows the transition from conventional fine-tuning to in-context learning and agentic reasoning modules. This research divides approach into three categories: agentic architectures with iterative reasoning and modification, prompt-based tactics, and refined LLM adaptations. These agentic frameworks show how hybrid RAG systems may combine relational databases with unstructured text. This combination provides context-sensitive responses by schema information. Also this can produce an execution feedback to analyze complex queries and improve SQL outputs [20].

2.2.2 The Challenge of “Dirty” Real-World Data

Text-to-SQL and structured retrieval by hybrid RAG systems have some challenges since common data in the world exists as noisy, partial, or unclear. A research indicates

that errors in natural language queries and SQL annotations are common in the BIRD-Bench dataset [21]. In addition, it demonstrates how model evaluations may be distorted by unclear language, grammatical errors, and inaccurate "pure" searches. This situation makes it challenging to assess real errors and dataset noise. This demonstrates the need for noise-aware evaluation, and the handling of bad data. Removing these problems can either greatly improve model performance or in some cases allow for simpler methods to do better than more complex ones [21].

Model effectiveness is impacted by annotation noise but also by user language difference. Furthermore, another study exhibit notable performance declines on Spider-Syn, a synonym-augmented version of the Spider dataset, indicating that neural Text-to-SQL models are susceptible to lexical mismatches and paraphrases [22]. This illustrates the more general difficulty of real-world inputs, where users frequently utilize informal phrases, paraphrases, or domain-relevant vocabulary that cannot be found in carefully selected training data [22].

To overcome these difficulties, models that use deeper embeddings or synonym-aware techniques must generalize semantically beyond exact token matches. This emphasizes the need of handling noisy annotation and text heterogeneity for hybrid RAG systems [21, 22].

2.2.3 Agentic Text-to-SQL and Self-Correction

Recent studies in Text-to-SQL with an emphasis on the role of agency in Text-to-SQL modeling started to move beyond the limitations of relying on static context by enabling models to dynamically model databases in a text-to-SQL setting and enabling error correction in a self-driven manner. SDE-SQL (Self-Driven Exploration SQL) which is a model that aims to leverage large models to self-explore databases by generating SQL probes for query generation as well as for iterations of query refinement, enabling a model to self-correct in a text-to-SQL setting without relying on structural information [23]. SDE-SQL models' databases using SQL probes provide information about databases and how to write accurate SQL queries in a zero-shot environment. This model gives users some control in a text-to-SQL setting. It does this by exploring databases in a self-driven manner. Moreover, it fixes any problems with query creation instead of external sources knowledge.

Experimental studies using benchmarks like BIRD provide evidence for SDE-SQL to outperform other models in text-to-SQL in terms of proper execution in a SQL setting, especially when databases are limited in size for proper query generation models [23]. The integration of exploration and refinement in a text-to-SQL model provides insights into how models, as agents in a text-to-SQL setting, provide form insights for hybrid RAG models in providing models with knowledge to correct themselves in a database setting rather than relying on humans for knowledge.

2.2.4 Handling Ambiguity and Interaction

Linguistic ambiguity in real-world natural language inquiries is frequently unresolvable in a single iteration. When many semantically different SQL searches might legitimately respond to the same query, such as when consumers ask, "Show me the top customers," without defining the measure of demand, ambiguity results. This is a basic difficulty for LLM-based Text-to-SQL systems, according to a study, which points out that existing datasets and benchmarks often ignore ambiguity and linguistic mismatch between user intent and database information [24]. Because there might not be a single definite SQL solution without further user explanation, this uncertainty makes evaluation and model accuracy more difficult [24].

BIRD-INTERACT, a benchmark and evaluation environment that treats Text-to-SQL tasks as interactive, multi-turn dialogues rather than static, one-shot challenges [25]. Models in BIRD-INTERACT have the ability to ask questions, access hierarchical database information. Also it makes independent decisions about whether to query a user simulator or investigate their fields. In order to create accurate, executable SQL, perplexing questions require follow-up inquiries and non-stop tweaking. This design acts as an actual production-grade requirements. The necessity of including multi-turn discussion and user input into hybrid RAG systems that manage ambiguity effectively is highlighted by empirical results on this benchmark, which demonstrate that even state-of-the-art LLMs perform badly without appropriate interaction mechanisms [25].

2.2.5 Industry Evaluation: BIRD vs. SPIDER

The two main benchmarks used by both academics and practitioners to evaluate Text-to-SQL systems' preparedness for industrial use are Spider and BIRD. Spider and BIRD both seek to assess semantic parsing proficiency. However, they reflect different paradigms in how they described the “real world”. This situation has a big impact on how models act and work for real-world settings. In the figure below, the main differences between these two evaluation models listed.

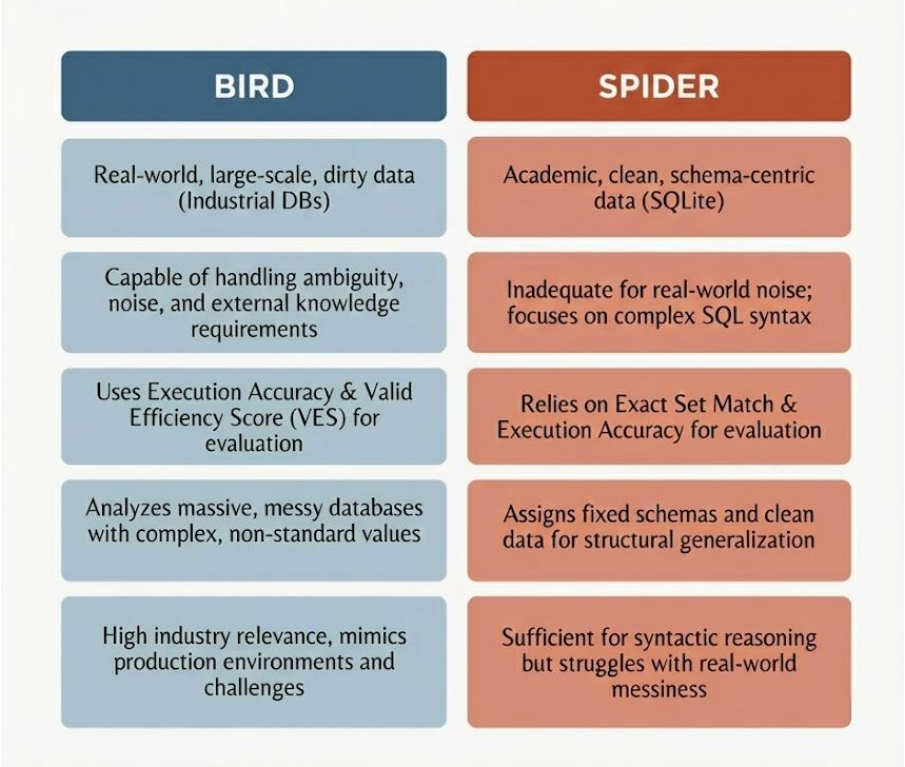


Figure 1. *BIRD vs Spider Comparison*

2.3 Hybrid Reasoning Frameworks

Hybrid reasoning has the potential of taking advantage of structured query processing as well as unstructured text retrieval. This makes RAG model to answer complex queries managing both databases and documents. In business settings, this is essential that choices are based on organized measurements and textual reports. Recent advances, such as data

linearization, neural-symbolic planning and iterative table conversion address the limitations of independent RAG or Text-to-SQL models by combining computational precision with semantic meaning. Hybrid frameworks provide context-aware, multiple-hop, scalable query responding across many data sources by merging reasoning paradigms.

2.3.1 Unifying Modalities via Linearization

UnifiedSKG, a method for hybrid reasoning that aims to combine structured and unstructured data by converting each assignment into a textual format [26]. UnifiedSKG enables a single Transformer model to process both textual passages and tabular data consistently by translating relational tables into serialized string expressions. This eliminates the need for distinct designs for each modality and allows for multi-task learning spanning information grounding, table reasoning, and question answering. By demonstrating that structured data may be embedded into the same latent space as natural language, the method simplifies hybrid reasoning pipelines by utilizing the expressive capacity of big sequence-to-sequence models to reason about heterogeneous inputs [26].

Large tables or databases cannot be completely serialized into a model's input, therefore linearization, despite its elegance, faces the context window bottleneck. A method named Fusion-in-Decoder (FiD) allows models to expand to bigger inputs while retaining generative reasoning power. This method achieves this encoding passages one by one and fusing representations in the decoder [27, 28]. Nevertheless, FiD continues to experience trouble with activities that call for complex aggregation or mathematical operations. Motivated by these constraints, hybrid architectures that combine RAG with linearized structured inputs show the ongoing trade-offs between unified text-based reasoning and strengths of structured query processing [26, 27].

2.3.2 Neural-Symbolic Reasoning: The Binder Approach

Neural-Symbolic Reasoning is a more reliable method for business systems by blending the adaptability of LLMs in terms of symbolic computing. The Binder framework is an example for this approach which views the LLM as a high-level system that generates executable programs (such as SQL or Python) with substitutions for lexical values To

answer a question like "Which country with a GDP over $1T has the highest happiness score?" Binder may, for instance, create an arbitrary SQL query that filters countries by GDP and use the LLM to extract the corresponding "happiness score" in unstructured textual sources. This division enables the neural engine to give semantic interpretation and context knowledge and the symbolic engine can carry out exact calculations along with logical filtering [29].

This bidirectional flow is crucial because the neural module manages dynamic semantic matching. Moreover, module includes paraphrase accuracy, and domain-specific vocabulary. Conversely, symbolic module handles deterministic logic and arithmetic. StructGPT extends this idea by making interfaces that allow the LLM to iteratively "read" relevant parts. Instead of relying on fully serialized representations, it checks graphs or databases. Through this step-by-step reading and reasoning process, the model can selectively access structured data sources. This leads to improved accuracy and greater efficiency in hybrid reasoning tasks [30].

Adaptively choosing the best reasoning route for each query, other research, like TableMaster, shows how LLMs may combine neural reasoning with symbolic operations in table-based contexts. TableMaster demonstrates the useful advantages of neural-symbolic hybrid architectures for enterprise-scale structured data interpretation by integrating table retrieval, enhanced verbalization, and program-aided reasoning [31]. Together, these frameworks demonstrate how flexible, comprehensible, and semantically sound solutions for complicated hybrid RAG systems are made possible by linking neural and symbolic reasoning.

Table 1. *Model – Limitation Comparison Table*

MODEL	TYPE	CAPABILITY	LIMITATION
BM25 [8]	Lexical Retrieval	Industry standard for probabilistic keyword matching (tf-idf).	Misses deep semantic connections; fails when queries lack exact keyword matches
DPR (DENSE PASSAGE RETRIEVAL) [11]	Neural Dense Retrieval	Encodes queries and texts into a common embedding space to capture semantic similarity.	Relies on large, supervised datasets making it expensive for domain-specific

			enterprise use where data is sparse.
STANDARD NEURAL TEXT-TO-SQL [21, 22]	Structured Retrieval	Translates natural language into executable SQL using benchmarks like Spider [18].	Susceptible to lexical mismatches, synonyms, and "dirty" real-world data (noise/paraphrasing)
UNIFIEDSKG [26]	Hybrid Reasoning (Linearization)	Unifies table and text reasoning by converting tables into linear text strings.	Large tables or databases cannot be completely serialized into the model's input.
FUSION-IN-DECODER (FID) [27, 28]	Hybrid Generative	Encodes passages/rows individually to handle larger inputs than linearization.	Struggles with tasks requiring deep aggregation or mathematical operations (sums, averages) where SQL excels.
NEURAL CONCEPT BINDER [29]	Neuro-Symbolic / Concept-Slot Encoding	Generates discrete concept representations from unsupervised data and uses retrieval-based "hard binding" to integrate external symbolic knowledge.	Unsupervised concept learning may produce incorrect concepts that require human revision; retrieval accuracy limits the effectiveness of the "hard binding" mechanism.
STRUCTGPT [30]	Iterative Reading-then-Reasoning (IRR)	Enhances LLM reasoning on large structured data (Tables, KGs, DBs) by using specialized interfaces to iteratively collect ("read") evidence.	Relies on the LLM's ability to correctly invoke interfaces (APIs); multiple iterative steps may increase latency compared to single-pass generation.

2.4 Additional Tools

This section describes four of the necessary tools selected to build the system. FAISS for vector retrieval, LangChain for orchestration, Google AI Studio for inference, and TableRAG for structured data reasoning.

2.4.1 High-Dimensional Vector Retrieval: FAISS

FAISS (Facebook AI Similarity Search) is developed by Meta Research to efficiently cluster and search for similarity of dense vectors. Unlike other libraries, it can search through sets of vectors of any size, even if they exceed the given RAM capacity. This makes FAISS crucial for any project's unstructured document storage or its caching system [32]. It utilizes indexing structures that are optimized which allow the system to compute dot products or Euclidean distances within milliseconds. This guarantees that the retrieval step of the operation does not become a bottleneck to the system [32].

2.4.2 Agent Orchestration Framework: LangChain

The project will include diverse components such as LLMs, SQL engines, and vector stores. To create an optimized workflow, LangChain will be utilized, helping to orchestrate this framework of components and agents. Handling the dual-path logic present in the project requires a framework such as LangChain to abstract certain concepts such as the conditional routing between the deep reasoning and the fast cache [6]. Furthermore, LangChain has multiple modules and toolkits that will enable the system to act as a cognitive runtime on its own by executing multi-step reasoning loops and maintaining a conversational state in itself [6].

2.4.3 Inference Environment: Google AI Studio

Google AI Studio will supply the system's primary functions using sophisticated models which have enormous context windows (up to 2 million tokens). The system can consume and reason over massive texts and datasets without experiencing fragmentation or context loss because to its architecture. When paired with its powerful Gemini API, Google AI Studio provides a complete development environment that enables quick engineering, iterative testing, and quick prototyping for those working on the project [33].

2.4.4 Structured Data Reasoning: TableRAG Framework

To handle complex database schemas efficiently, the project will adopt the TableRAG framework. Since traditional methods of Text-to-SQL struggle when the

number of tables grows large, the TableRAG framework fits the project scenario perfectly. TableRAG uses a retrieval-augmented strategy to first identify the schema, and then "prune" it. This allows the system to present only the columns relevant to the specific query to the LLM [5]. By employing such an approach, the hallucination rate and the token usage will be minimized when generating the SQL.

3.1 System Layers and Workflow Diagram

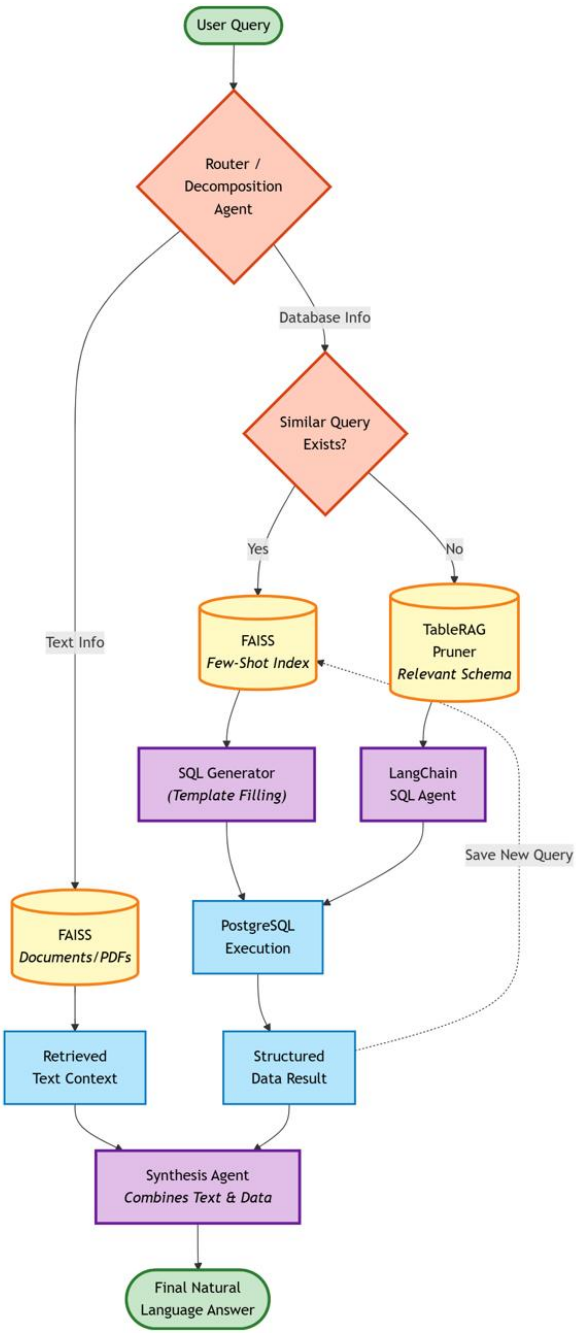


Figure 3. Adaptive Agentic RAG System Workflow Diagram showing the interaction between the Routing, Retrieval, and Synthesis layers [34]

Building upon the analysis of recent advancements in dense retrieval, neural-symbolic reasoning, and agentic workflows, this project proposes an Adaptive Agentic

RAG Framework. Unlike standard RAG implementations that treat retrieval as a static step, our methodology treats retrieval as a dynamic, agent-planned process.

The architecture will operate in three distinct layers:

Routing Layer: An intelligent system will classify the user's query to determine if it needs to use structured RAG or unstructured RAG. The query will be analyzed to decide if it needs to check SQL for a response or if it can be obtained by using default embeddings and an LLM model.

Retrieval Layer: A vector store like FAISS or pgvector will be used to hold vectors and perform semantic search on them.

The Synthesis Layer: A final part will generate a response according to the gathered data from the structured or unstructured parts.

3.1.1 Routing Layer

Routing layer is responsible for determining whether the user prompt needs to pull information from structured data or unstructured data. When a user submits a query, an LLM model with a smaller footprint compared to GPT-5.2 or similar versions will analyze the query and decide whether it requires unstructured retrieval, structured retrieval, or a hybrid of both.

Queries that require filtering, aggregation, or comparison of data will proceed to the SQL route, while descriptive information and summaries will be directed to the unstructured retrieval route.

3.1.2 Unstructured Layer

If the user's text pertains exclusively to qualitative information, such as summaries or textual descriptions, and does not need SQL data or other database access, the system will proceed to the unstructured retrieval layer.

This layer operates as a modern RAG system that uses dense retrieval and hybrid search among the system's designated vector data.

The system will convert user queries into vectors using an embedding model like multilingual-e5-base or all-MiniLM. The multilingual-e5-base model, which supports several languages for worldwide applications and maps text to a 768-dimensional dense vector space, is very reliable [35]. To properly handle asymmetric search operations, it uses "weakly supervised contrastive pre-training" and needs certain prefixes, such as "query: " for inputs and "passage: " for documents [36].

A FAISS index containing pre-computed embeddings of PDFs, docs, and other documents will be loaded into memory for hybrid search [4]. The system will use cosine similarity between the user's query and the documents in the FAISS index to retrieve the top-k most relevant text chunks.

3.1.3 Structured Retrieval Layer

If the routing layer decides the user query requires database records, the system activates the Structured Retrieval Layer. Instead of relying solely on standard SQL agents, our project employs a secondary routing mechanism to balance speed, accuracy, and LLM token usage.

3.1.3.1 Few-Shot Example (Fast Track)

The "Few-Shot Example" component is a distinct vector database within our system that contains pre-defined questions paired with the correct SQL commands to gather information. Before attempting to generate a new SQL query from the beginning, the system performs a hybrid search across the Faiss vector database for SQL queries that could be similar to user queries. These serve as golden examples that instruct LLM how to generate user instructions without using the agent.

When the system finds a similar hit from the vector database, it pulls the relevant SQL template and updates it with the new details (for example, swapping "Name X" for "Name Y"). This fast tracks the agent route, it is faster and more efficient resulting in less token generation and fewer LLM calls.

3.1.3.2 TableRAG and ReAct Agent

If the user's query cannot be found in the vector database, the system goes through the agent deep reasoning path.

Schema Pruning (TableRAG): SQL agents generally request a fetch for the entire database schema to understand the structure. However, big SQL schemas can overwhelm the LLM and result in hallucinations [5].

To prevent this, we perform a third semantic search to identify only the tables relevant to the user's query. This approach cuts the cost of agentic loops required to gather table information and allows the SQL agent to proceed directly to generating a query.

LangChain SQL Agent: For this purpose, we will employ a LangChain SQL Agent since it has several benefits: it can run queries on its own, make judgments, and self-correct mistakes, such managing inaccurate data returns or resolving invalid SQL instructions. According to research, using such agentic workflows, more especially, those with mistake correction and iterative reasoning capabilities, significantly improves execution accuracy and resilience compared to static sequential query generation techniques [37].

3.1.4 Synthesis Layer

Synthesis layer is responsible for generating the final response from the retrieved data. At the final stage, an LLM model will check the correctness of the gathered information and generate a response based on whether the query followed a hybrid, structured, or unstructured route. The synthesis layer takes all the retrieved information and generates statistical summaries or aggregated values according to the user’s commands.

If the evidence is incomplete, incorrect, or contains conflicts, the model will proceed accordingly and ask the user for clarification instead of producing a potentially misleading answer.

3.2 System Architecture Overview

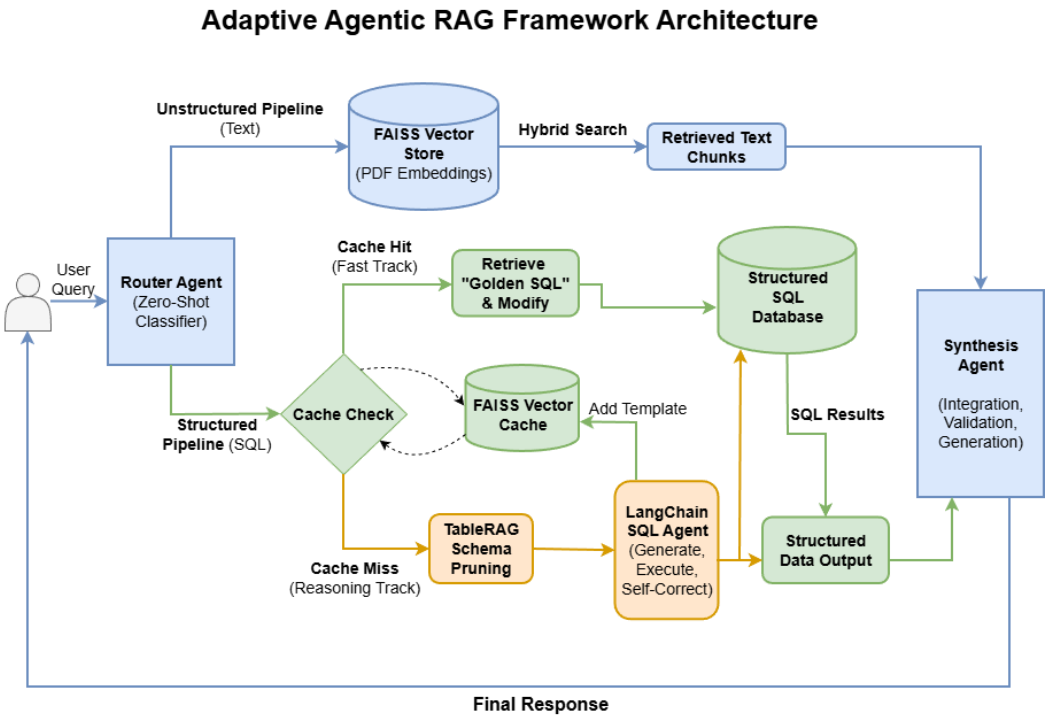


Figure 4. Adaptive Agentic RAG Framework Architecture diagram

As can be seen from the diagram on Figure 4, the proposed system architecture shows how different components of the system work together to answer the prompt of the user in the best possible way. When a prompt comes, Router Agent decides which pipeline should the information come from based on the analysis of the prompt and its sub-prompts derived from the agent. This can either be forwarded to structured pipeline, unstructured pipeline or from both. For questions that require the need to pull structured information, the system first checks if a similiar question exists in the vector database; if there is a similar question existing, the system will use that query quickly. If not, the system switches to a mode where a more advanced agent explores the database and creates a schema-aware, correct SQL query. Meanwhile, the prompts that require unstructured pipeline (text-based), works like a traditional RAG where a FAISS search is done from stored documents inside the vector database for vector similarity to find the most relevant pieces of information. Finally, at the end of the process, both outputs from unstructured pipeline and structured pipeline are combined for a final response that is clear and correct (Figure 4).

WORK PACKAGE DESCRIPTION

Number and name of the package: Work Package I: Intelligent Routing and Unstructured Retrieval

Start date: 01-01-2026

Finish date: 10-02-2026

Short description: We aim to build the system's input processing layer by developing an intelligent router to classify user intents and setting up a vector-based retrieval pipeline for unstructured document handling.

Methodology: The work begins by developing a Zero-Shot LLM classifier using LangChain to categorize user queries into structured or unstructured intents. Simultaneously, we will collect and preprocess the project's PDF datasets using recursive character splitting. These text chunks will be embedded using a transformer model and indexed into a FAISS vector database to enable efficient dense retrieval for document-based questions.

Outputs: Functioning Router Agent, populated FAISS Vector Index, and Python retrieval scripts.

Relations to other packages: Provides the necessary routing logic for WP II and retrieves text context for WP III.

Working team members (with contributions as percentages of the work packet):
Erk Demirel 25%, Emir Özen 25%, Mehmet Ekin Ay 25%, Volkan Erdoğan 25%.

Work to be done by any body outside the team: -

WORK PACKAGE DESCRIPTION

Number and name of the package: Work Package II: Structured Retrieval Layer & Dual Path

Start date: 01-02-2026

Finish date: 20-03-2026

Short description: We aim to implement the "Dual-Path" SQL reasoning engine that dynamically switches between a fast caching mechanism and a deep reasoning agent for database interactions.

Methodology: A secondary FAISS index will be initialized to serve as a "Golden SQL" cache for few-shot retrieval of repetitive queries. For complex queries that miss the cache, we will integrate the TableRAG framework to perform semantic schema pruning. This pruned schema will then be passed to a LangChain SQL Agent (ReAct) which will autonomously generate and execute SQL queries against the relational database.

Outputs: Functional Dual-Path SQL pipeline, TableRAG schema pruning module, and SQL Agent integration.

Relations to other packages:receives inputs from WP I and sends SQL results to WP III.

Working team members (with contributions as percentages of the work packet):
Erk Demirel 25%, Emir Özen 25%, Mehmet Ekin Ay 25%, Volkan Erdoğan 25%.

Work to be done by any body outside the team: -

WORK PACKAGE DESCRIPTION

Number and name of the package: Work Package III: Synthesis Layer and Interface Development

Start date: 15-03-2026

Finish date: 25-04-2026

Short description: We aim to merge the retrieved data into a coherent natural language response and develop a user-friendly web interface to visualize the system's reasoning and results.

Methodology: A Synthesis Agent will be developed to combine text chunks from the unstructured layer and SQL results from the structured layer into a final answer with citations. Following this, a full-stack web interface will be built using Streamlit or Chainlit. This interface will be connected to the backend pipelines to display the chat history, reasoning steps, and source documents to the user.

Outputs: End-to-end RAG system, Synthesis Answer Module, and Web User Interface.

Relations to other packages: Consumes outputs from WP I and WP II; prerequisite for validation in WP IV.

Working team members (with contributions as percentages of the work packet):
Erk Demirel 25%, Emir Özen 25%, Mehmet Ekin Ay 25%, Volkan Erdoğan 25%.

Work to be done by any body outside the team: -

WORK PACKAGE DESCRIPTION

Number and name of the package: Work Package IV: Testing, Validation, & Reporting

Start date: 20-04-2026

Finish date: 15-05-2026

Short description: We aim to validate the system's performance through rigorous testing and compile the final graduation thesis and presentation materials.

Methodology: The latency and accuracy differences between the cache-based and agent-based pathways will be measured using a test set of intricate queries. To confirm the architectural efficiency, the outcomes will be examined. Lastly, the final report, presentation and demo will be developed, and the graduation thesis will cover the whole system design, implementation specifics, and results from experiments.

Outputs: Performance Evaluation Report, Final Graduation Thesis, and Project Presentation.

Relations to other packages: Validates the complete system built in WP III.

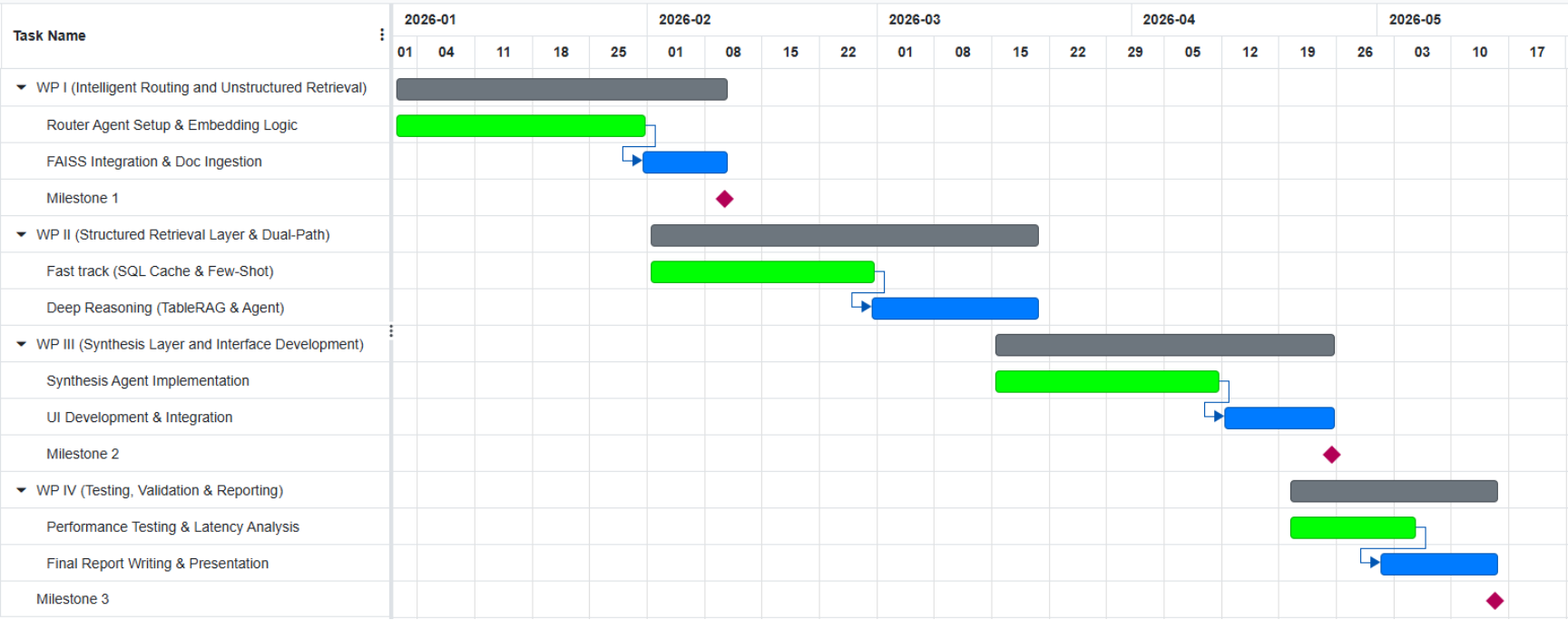
Working team members (with contributions as percentages of the work packet):
Erk Demirel 25%, Emir Özen 25%, Mehmet Ekin Ay 25%, Volkan Erdoğan 25%.

Work to be done by any body outside the team: -

CONTRIBUTIONS OF THE TEAM MEMBERS:

Name, Surname	Work Packages								Total	
	I		II		III		IV			
	h	%	h	%	h	%	h	%	h	%
Erk Demirel	45	25	55	25	45	25	30	25	**175**	**25**
Emir Özen	45	25	55	25	45	25	30	25	**175**	**25**
Mehmet Ekin Ay	45	25	55	25	45	25	30	25	**175**	**25**
Volkan Erdoğan	45	25	55	25	45	25	30	25	**175**	**25**
Total	**180**	**100**	**220**	**100**	**180**	**100**	**120**	**100**	**700**	**100**

PROJECT CALENDAR



INFRA-STRUCTURE REQUIREMENTS:

Laboratory(ies): -

Equipment: -

Computer(s): -

Personnel: -

PROJECT BUDGET:

This study will not utilize any funds for any of the following: transportation, services, publications/software, equipment, or consumables.

	Work Packages				
	I	**II**	**III**	**IV**	**TOTAL**
Equipment	-	-	-	-	-
Consumable goods	-	-	-	-	-
Publications/Software	-	-	-	-	-
Transportation	-	-	-	-	-
Services	-	-	-	-	-
TOTAL	-	-	-	-	0

Equipment: -

Consumable goods: -

Publications/Software: -

Transportation: -

Services: -

REFERENCES:

[1] Minaee, S., Mikolov, T., Nikzad, N., Chenaghlu, M., Socher, R., Amatriain, X., & Gao, J., "Large Language Models: A Survey," *arXiv preprint*, 2024. Available: https://arxiv.org/pdf/2402.00013

[2] Bommasani, R., Hudson, D., Adeli, E., et al., "On the Opportunities and Risks of Foundation Models," *arXiv preprint*, 2021. Available: https://arxiv.org/pdf/2108.07258

[3] Lewis, P., Perez, E., Piktus, A., et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems*, pp. 9459–9474, 2020. Available: https://arxiv.org/pdf/2005.11401

[4] Facebook AI Research, "Faiss: A Library for Efficient Similarity Search and Clustering of Dense Vectors," 2024. Available: https://faiss.ai

[5] X. Yu, P. Jian, and C. Chen, "TableRAG: A Retrieval Augmented Generation Framework for Heterogeneous Document Reasoning," arXiv preprint arXiv:2506.10380, 2025. Available: https://arxiv.org/html/2506.10380v2.

[6] Chase, H., "LangChain," 2022. Available: https://www.langchain.com/langchain

[7] Gao, Y., Xiong, Y., Gao, X., et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv preprint*, 2024. Available: https://arxiv.org/pdf/2312.10997

[8] Robertson, S., and Zaragoza, H., "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, pp. 333–389, 2009. Available: https://www.researchgate.net/publication/220613776_The_Probabilistic_Relevance_Framework_BM25_and_Beyond#fullTextFileContent

[9] Kamphuis, C. F. H. et al., "Which BM25 Do You Mean? A Large-Scale Reproducibility Study of Scoring Variants," *European Conference on Information Retrieval*, 2020. Available: https://www.researchgate.net/publication/340573528_Which_BM25_Do_You_Mean_A_Large-Scale_Reproducibility_Study_of_Scoring_Variants#fullTextFileContent

[10] Guo, J. et al., "Semantic Models for the First-stage Retrieval: A Comprehensive Review," *ACM Transactions on Information Systems*, 2021. Available: https://arxiv.org/pdf/2103.04831

[11] Karpukhin, V. et al., "Dense Passage Retrieval for Open-Domain Question Answering," *EMNLP*, 2020. Available: https://aclanthology.org/2020.emnlp-main.550.pdf

[12] Salahmah, A. H., McWhannel, P., and Yan, N., "Dense Passage Retrieval in Conversational Search," *arXiv preprint*, 2025.Available: https://arxiv.org/pdf/2503.17507

[13] Izacard, G., Caron, M., Hosseini, L., Riedel, S., Bojanowski, P., Joulin, A., & Grave, E., "Unsupervised Dense Information Retrieval with Contrastive Learning," *OpenReview*, 2022. Available: *https://openreview.net/pdf?id=jKN1pXi7b0*

[14] Gao, L., Ma, X., Lin, J., & Callan, J., "Precise Zero-Shot Dense Retrieval without Relevance Labels," *arXiv preprint*, 2022. Available: *https://arxiv.org/pdf/2212.10496*

[15] DeepSet, "Hypothetical Document Embeddings (HyDE)," Haystack Documentation, 2025. Available: https://docs.haystack.deepset.ai/docs/hypothetical-document-embeddings-hyde

[16] Khattab, O. and Zaharia, M., "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," in *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '20)*, Virtual Event, China, 2020, pp. 39–48, doi: 10.1145/3397271.3401075.

[17] Li, M., Lin, S.-C., Ma, X., and Lin, J., "SLIM: Sparsified Late Interaction for Multi-Vector Retrieval with Inverted Indexes," *arXiv preprint arXiv:2302.06587*, 2023, Available: https://arxiv.org/abs/2302.06587

[18] Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., & Radev, D., *"Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task,"* in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP 2018)*, Brussels, Belgium, 2018, pp. 3911–3921. Available: https://aclanthology.org/D18-1425.pdf

[19] Zhu, X., Li, Q., Cui, L., & Liu, Y., *"Large Language Model Enhanced Text-to-SQL Generation: A Survey,"* arXiv preprint, 2024. Available: https://arxiv.org/pdf/2410.06011

[20] Gorti, S. K., Gofman, I., Liu, Z., Wu, J., Vouitsis, N., Yu, G., Cresswell, J. C., & Hosseinzadeh, R., *"MSc-SQL: Multi-Sample Critiquing Small Language Models for Text-to-SQL Translation,"* arXiv preprint, 2025. Available: https://arxiv.org/pdf/2410.12916v2

[21] Wretblad, A., Zeng, Z., & Kratzwald, B., *"BIRD-Bench: Evaluating the Impact of Noisy Queries and SQL Annotations on Text-to-SQL Models,"* in *Proceedings of the 2024 Annual Meeting of the Association for Computational Linguistics (ACL 2024 Short Papers)*, Santa Clara, USA, 2024, pp. 300–312. Available: https://aclanthology.org/2024.acl-short.34.pdf

[22] Gan, Z., Sun, Y., Zhang, X., & Wong, K. F., *"Spider-Syn: Evaluating Neural Text-to-SQL Models on Lexical Variation and Synonym Robustness,"* in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL 2021 Long Papers)*, Online, 2021, pp. 7453–7465. Available: https://aclanthology.org/2021.acl-long.195.pdf

[23] Xie, Y., Li, H., Zhang, R., & Wang, Q., *"SDE-SQL: Agentic Text-to-SQL with Self-Driven Exploration and Correction,"* arXiv preprint, 2025. Available: https://arxiv.org/pdf/2506.07245

[24] Shi, L., Tang, Z., Zhang, X., & Yang, Z., *"A Survey on Employing Large Language Models for Text-to-SQL Tasks,"* arXiv preprint, 2024. Available: https://arxiv.org/html/2407.15186v5

[25] Huo, N., Xu, X., Li, J., Jacobsson, P., Lin, S., Qin, B., Hui, B., Li, X., Qu, G., Si, S., Han, L., Alexander, E., Zhu, X., Yu, R., Jin, Y., Zhou, F., Zhong, W., Chen, Y., Liu, H., Ma, C., Ozcan, F., Papakonstantinou, Y., & Cheng, R., *"BIRD-INTERACT: Re-imagining Text-to-SQL Evaluation for Large Language Models via Lens of Dynamic Interactions,"* arXiv preprint, 2025. Available: https://arxiv.org/pdf/2510.05318

[26] Xie, R., Chen, Q., Chen, W., & Sun, M., "UnifiedSKG: A Unified Model for Structured Knowledge Grounding," arXiv preprint, 2022. Available: https://arxiv.org/pdf/2201.05966

[27] Izacard, G., & Grave, E., “Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering,” arXiv preprint, 2020. Available: https://arxiv.org/pdf/2007.01282

[28] Izacard, G., Dou, Z., & Grave, E., “Fusion-in-Decoder: Efficient Multi-Passage Reading for Generative Models,” ACL Long Papers, 2023. Available: https://aclanthology.org/2023.acl-long.454

[29] Stammer, W., Wüst, A., Steinmann, D., & Kersting, K., “Neural Concept Binder,” arXiv preprint, 2024. Available: https://arxiv.org/abs/2406.09949

[30] Jiang, J., Zhou, K., Dong, Z., Ye, K., Zhao, W. X., & Wen, J.-R., “StructGPT: A General Framework for Large Language Model to Reason over Structured Data,” *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 9237–9251, 2023. Available: https://aclanthology.org/2023.emnlp-main.574/

[31] Cao, L., “TableMaster: A Recipe to Advance Table Understanding with Language Models,” arXiv preprint, 2025. Available: https://arxiv.org/abs/2501.19378

[32] Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P.-E., Lomeli, M., Hosseini, L., & Jégou, H., “The Faiss library,” *arXiv preprint*, 2024. Available: https://arxiv.org/abs/2401.08281

[33] Google, “Long context | Gemini API,” *Google AI for Developers*, 2024. Available: https://ai.google.dev/gemini-api/docs/long-context

[34] Mermaid, “Mermaid: Diagramming and charting tool,” https://mermaid.js.org/ , accessed Mar. 2025.

[35] Dataloop, “Sentence Transformers Multilingual E5 Base,” *Dataloop AI Library*, 2023. Available: https://dataloop.ai/library/model/embaas_sentence-transformers-multilingual-e5-base/

[36] Patel, A., “The Practitioner's Guide To E5,” *Pinecone Learn*, 2024. Available: https://www.pinecone.io/learn/the-practitioners-guide-to-e5/

[37] Fan, C.-Y., & Chen, J.-C. (2024). "Development and Evaluation of Learning Portfolio Query System Based on LangChain Framework." *Engineering Proceedings*, 92(1), 40. MDPI. Available at: https://www.mdpi.com/2673-4591/92/1/40